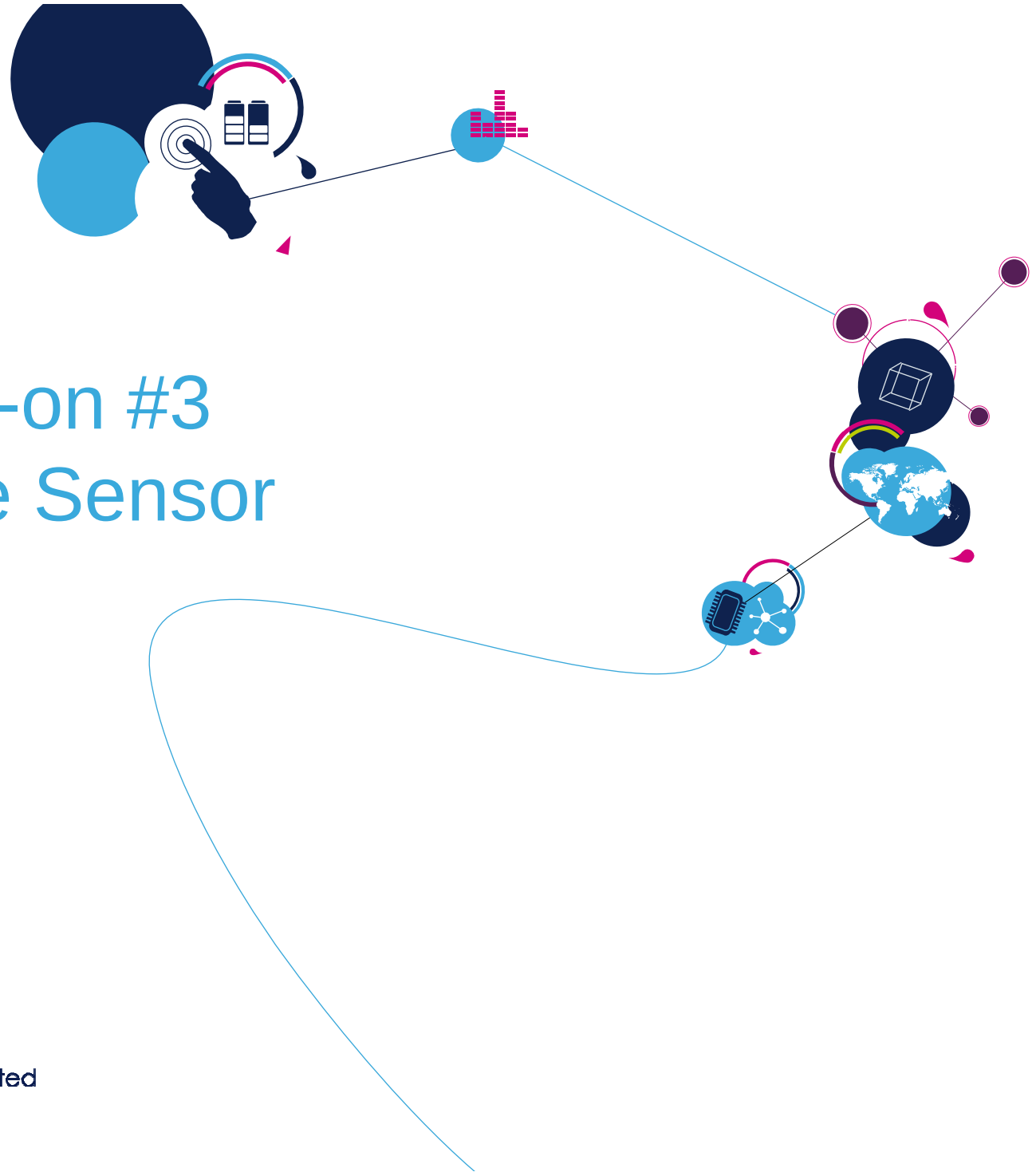


LoRa hands-on #3

Temperature Sensor



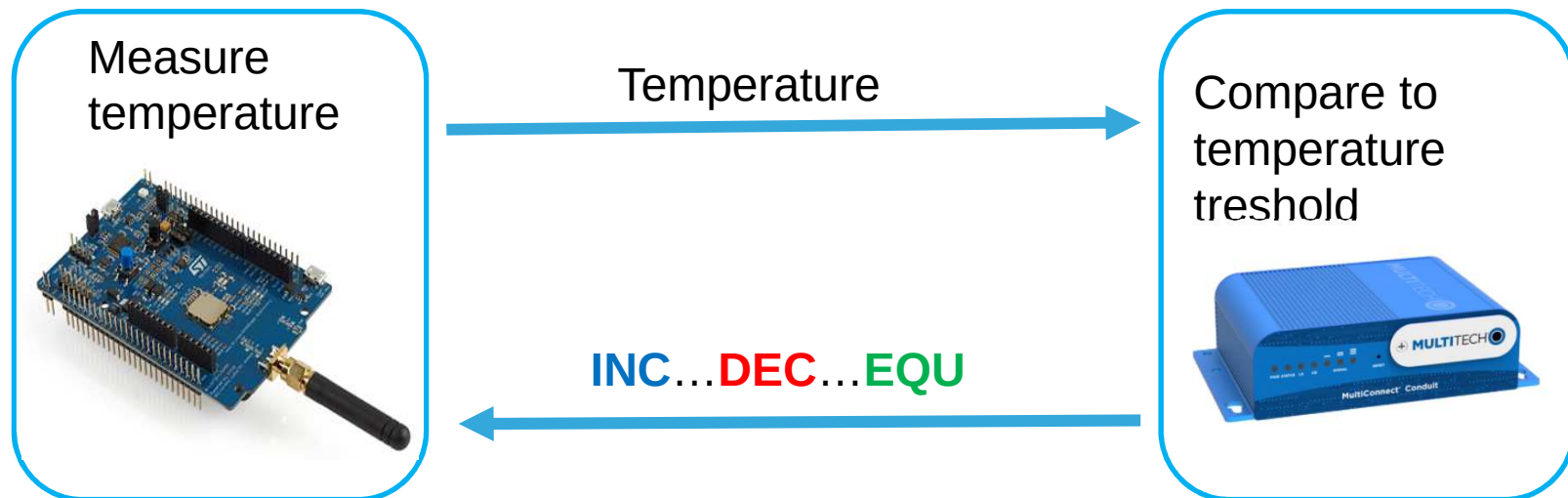
Hands-on

- Key learning

- Sending and receiving data from application layer using user defined actions both in RX and TX data call backs.
- Handling of application data buffer.
- User definition of software timer to indicate any event using LED.
- Using of hw functions of I-CUBE-LRWAN: MCU internal temperature sensor handling
- Debug of application

Hands-on

Temperature sensor application



Hands-on

Temperature sensor application

1. Mote measures temperature using internal STM32L072 temperature sensor and sends back value to gateway every 60s.
2. Gateway compares received temperature value to internal threshold and send response to mote: (I)ncrease if $T < T_{rsh}$, (D)ecrease if $T > T_{rsh}$, (E)qual if $T = T_{rsh}$
3. If mote receives (E)qual response green led lights, else blue or red led lights depends on (I)ncrease or (D)ecrease response has been received

Hands-on

STM32L072 temperature sensor

There is internal temperature sensor connected to internal ADC peripheral (ADC_IN18), factory-calibrated for 2 temperatures: 30°C and 130°C, calibration data are stored in MCU system memory area

Calibration value name	Description	Memory address
TS_CAL1	TS ADC raw data acquired at temperature of 30 °C, $V_{DDA}=3\text{ V}$	0x1FF8 007A - 0x1FF8 007B
TS_CAL2	TS ADC raw data acquired at temperature of 130 °C, $V_{DDA}=3\text{ V}$	0x1FF8 007E - 0x1FF8 007F

Symbol	Parameter	Min	Typ	Max	Unit
$T_I^{(1)}$	V_{SENSE} linearity with temperature	-	± 1	± 2	°C
Avg_Slope ⁽¹⁾	Average slope	1.48	1.61	1.75	mV/°C
V_{130}	Voltage at 130°C $\pm 5^\circ\text{C}^{(2)}$	640	670	700	mV
$I_{DDA(TEMP)}^{(3)}$	Current consumption	-	3.4	6	μA
$t_{START}^{(3)}$	Startup time	-	-	10	μs
$T_{S_temp}^{(4)(3)}$	ADC sampling time when reading the temperature	10	-	-	

Hands-on

STM32L072 temperature sensor

COMPUTE_TEMPERATURE macro defined in **mlm32l0xx_hw.c** and stored temperature calibration values are included in computation. Mentioned macro is used inside **HW_GetTemperatureLevel()** which can be call to get temperature value in [°C].

```
82  /* Internal temperature sensor: constants data used for indicative values in */
83  /* this example. Refer to device datasheet for min/typ/max values.          */
84
85  /* Internal temperature sensor, parameter TS_CAL1: TS ADC raw data acquired at
86   * a temperature of 30 DegC (+-5 DegC), VDDA = 3.0 V (+-10 mV). */
87  #define TEMP30_CAL_ADDR ((uint16_t*) ((uint32_t) 0xFF8007A))
88
89  /* Internal temperature sensor, parameter TS_CAL2: TS ADC raw data acquired at
90   * a temperature of 130 DegC (+-5 DegC), VDDA = 3.0 V (+-10 mV). */
91  #define TEMP130_CAL_ADDR ((uint16_t*) ((uint32_t) 0xFF8007E))
92
93  /* Vdda value with which temperature sensor has been calibrated in production
94   * (+-10 mV). */
95  #define VDDA_TEMP_CAL ((uint32_t) 3000)
96
97
98  #define COMPUTE_TEMPERATURE(TS_ADC_DATA, VDDA_APPLI)
99  (((((( (int32_t) ((TS_ADC_DATA * VDDA_APPLI) / VDDA_TEMP_CAL)
100   - (int32_t) *TEMP30_CAL_ADDR)
101   ) * (int32_t) (130 - 30)
102   ) << 8) / (int32_t) (*TEMP130_CAL_ADDR - *TEMP30_CAL_ADDR)
103   ) + (30 << 8)
104  )
```

Hands-on

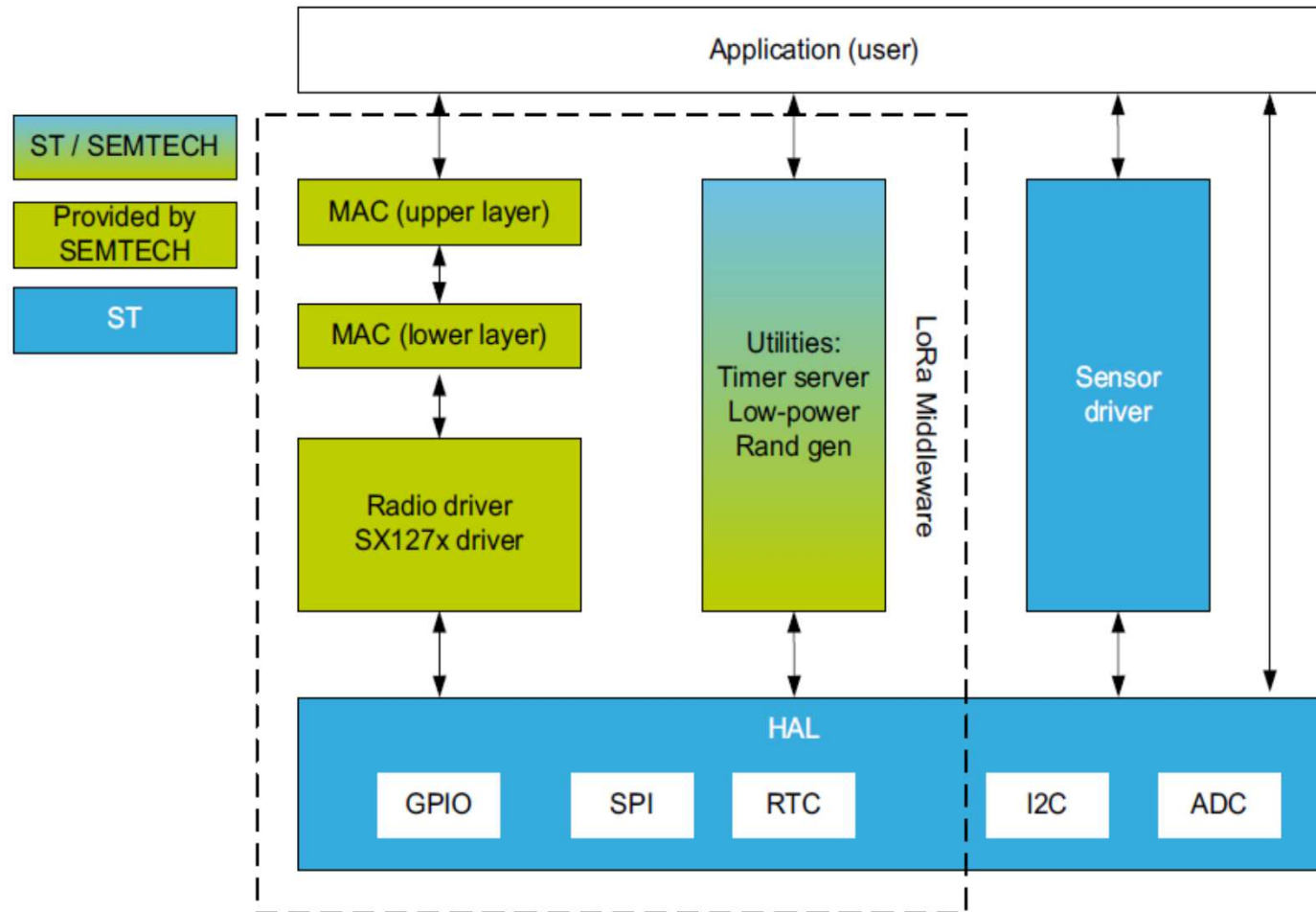
Architecture

Hints:

Detailed description in UM2073: “STM32 LoRa® software expansion for STM32Cube”

Hands-on

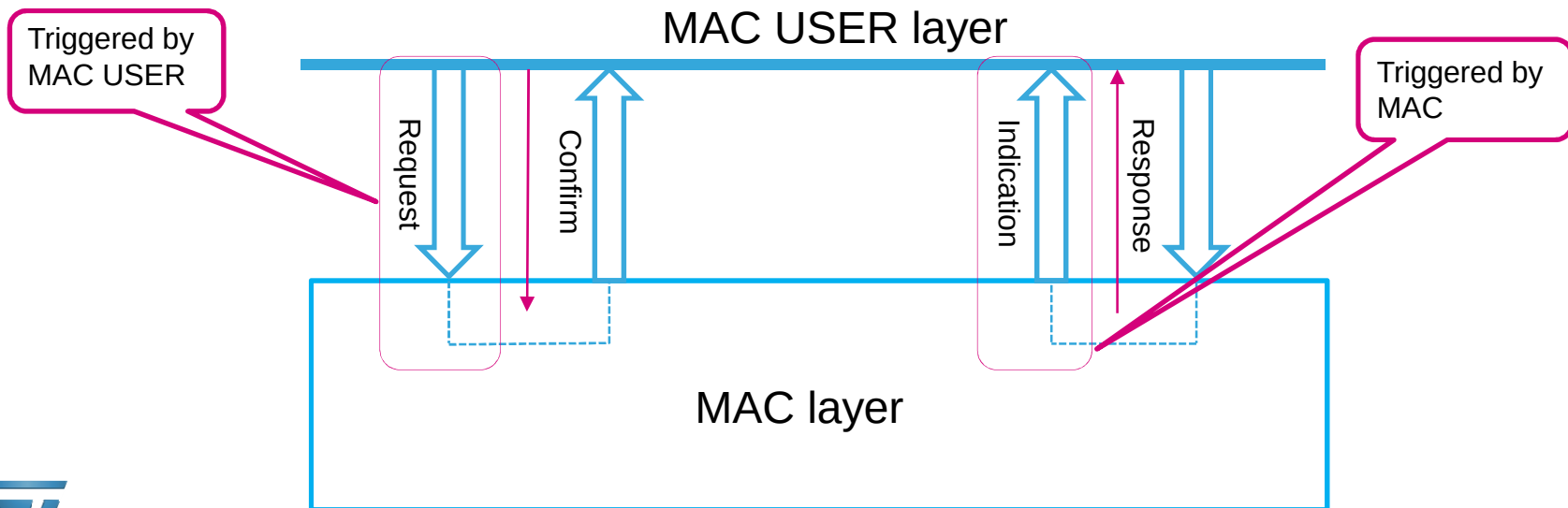
I-CUBE-LRWAN basics: architecture



IEEE 802.15.4 introduction: MAC API

I-CUBE-LRWAN protocol stack implementation base on IEEE 802.15.4 standard. Communications are passed between the MAC layer and the next highest stack layer (MAC user) by means of “service primitives”

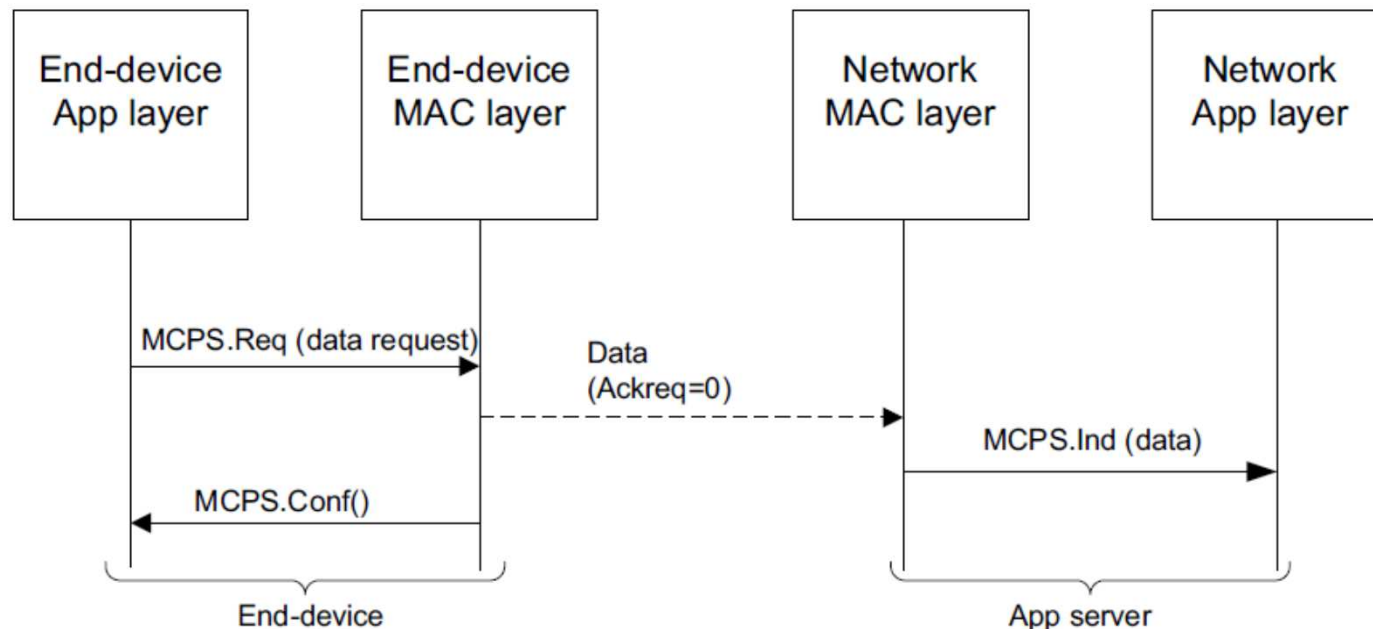
- Request ; initiated by the MAC user
- Confirm ; MAC user may solicit Confirm from the MAC layer
- Indication ; initiated by the MAC layer
- Response ; MAC layer may solicit Response from the MAC user



IEEE 802.15.4 introduction: MAC services

IEEE 802.15.4 MAC provides the interface to the application layer using two elements:

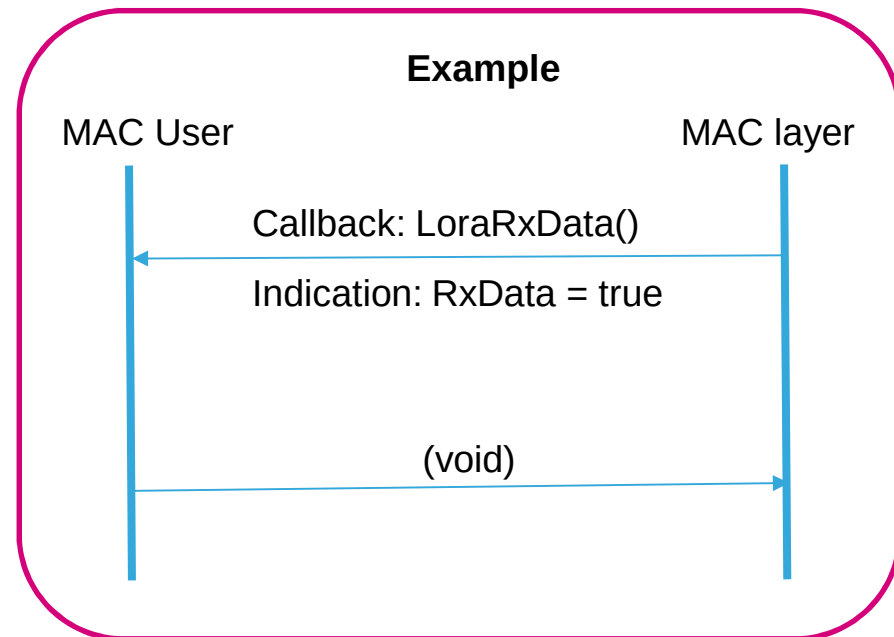
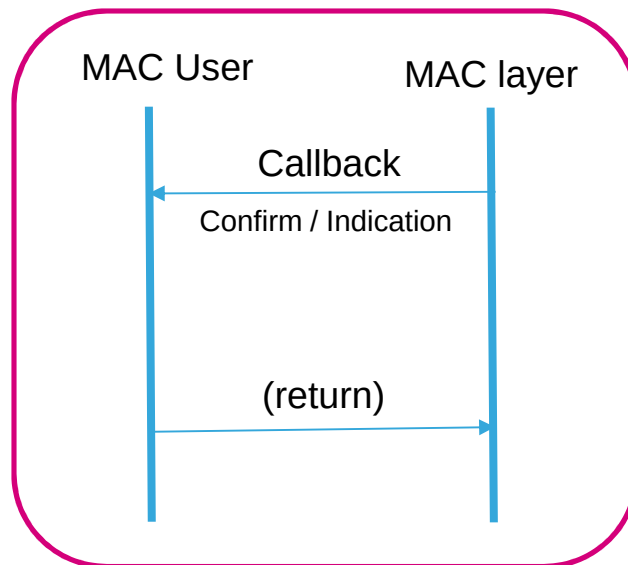
- MAC Management Service: this is called **MAC Layer Management Entity**, MLME. It provides the service through which layer management functions may be called or accessed. Example: JOIN network REQUEST,
- MAC Data Service: this is called the **MAC Common Port Service**, MCPS. It provides data transport services. Example: SEND data REQUEST.



Hands-on

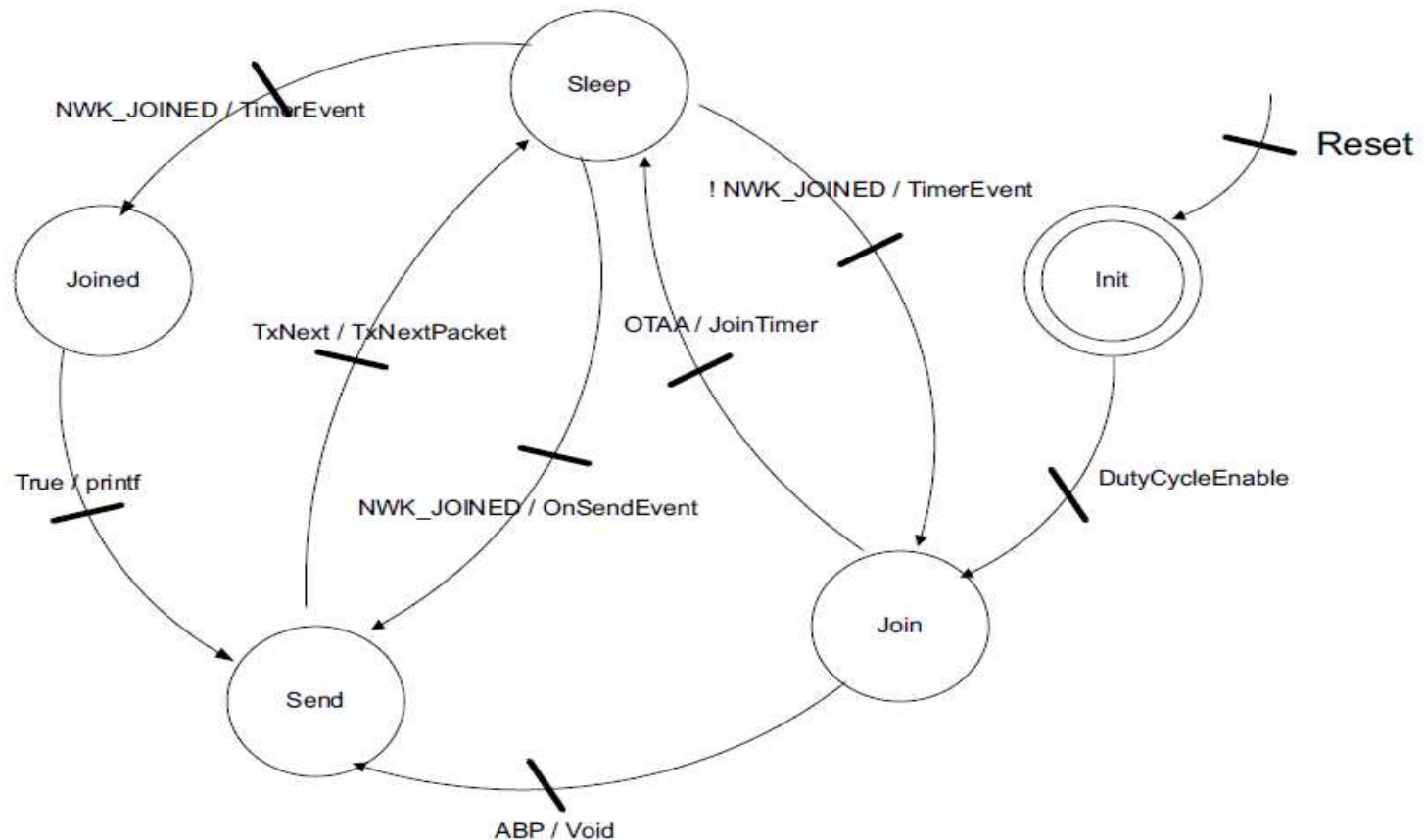
IEEE 802.15.4 introduction: callback mechanism

Request is issued by the MAC User by means of call to one of the MAC layer API functions. The most straightforward way for the MAC layer to reply (with Confirm and/or Indication) is via callback function.



I-CUBE-LRWAN basics: operation model

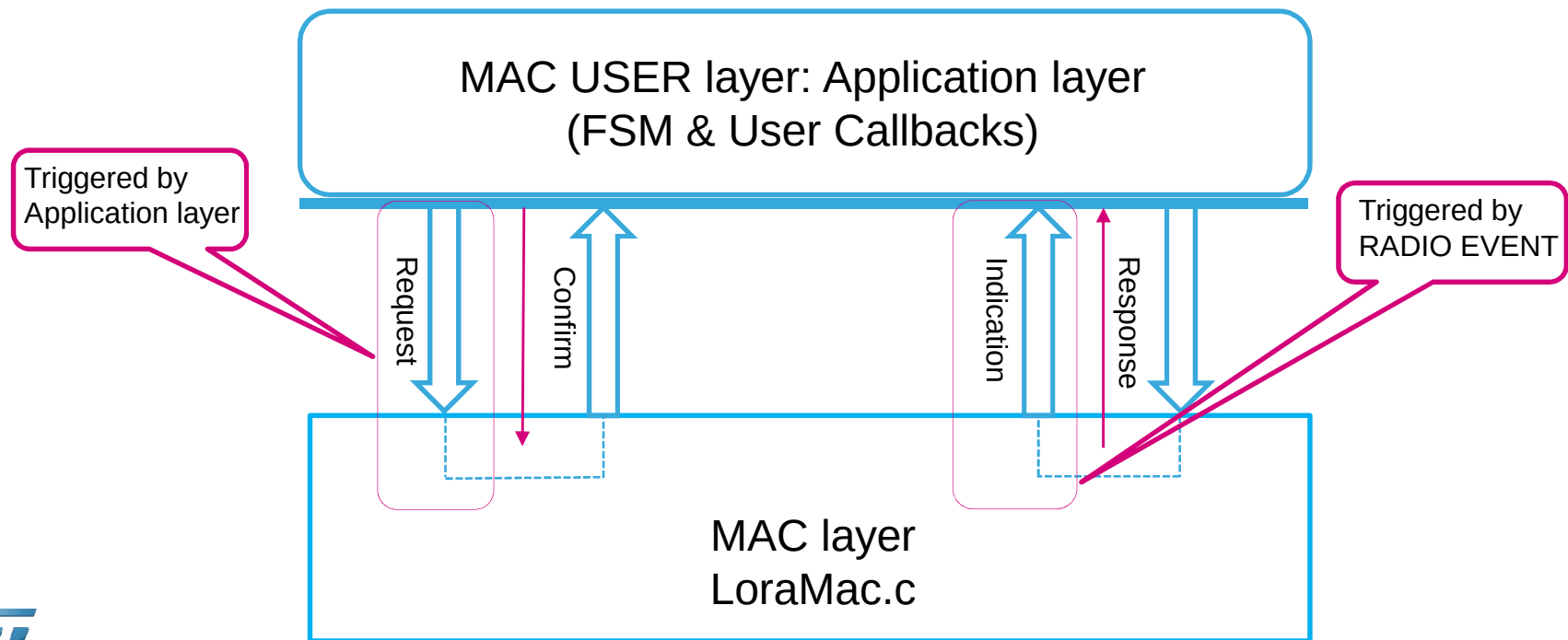
The model of operation proposed for this LoRa class A is based on **Finite State Machine (FSM)** running in the main thread of the application. LoRa FSM transition is triggered either by a timer event (Request) or by a radio event (Indication).



I-CUBE-LRWAN basics: operation model

IEEE 802.15.4 and operation model in practice:

- FSM requests services of MAC layer: Request / Confirm mechanism (i.e. send data)
- MAC layer indicates to FSM radio events and invokes related user callbacks (i.e. receive data event)



I-CUBE-LRWAN basics: fsm implementation

LoraWAN FSM is implemented in lora.c / lora.h source files

1. Initialization of the Lora class A FSM

```
void lora_Init (LoRaMainCallback_t *callbacks,  
LoRaParam_t* LoRaParamInit)
```

2. Lora class A FSM entry point

```
void lora_fsm (void)
```

3. Make Tx frame callback function, user is free to implement own code here

```
void LoraTxData (lora_AppData_t *AppData,  
FunctionalState* IsTxConfirmed)
```

4. Make Rx frame callback function, user is free to implement own code here

```
void LoraRxData (lora_AppData_t *AppData)
```

I-CUBE-LRWAN basics: fsm implementation

LoraWAN FSM is implemented in lora.c / lora.h source files

1. Get of the Lora class A FSM current state

```
DeviceState_t lora_getDeviceState( void );
```

Lora FSM states definition

```
typedef enum eDevicState  
{  
    DEVICE_STATE_INIT,  
    DEVICE_STATE_JOIN,  
    DEVICE_STATE_JOINED,  
    DEVICE_STATE_SEND,  
    DEVICE_STATE_CYCLE,  
    DEVICE_STATE_SLEEP  
} DeviceState_t;
```

I-CUBE-LRWAN basics: software timers

All the time dependences of I-CUBE-LRWAN are realized using software timer server `timeServer.c / timeServer.h`

1. Initialize the timer and associate callback

```
void TimerInit (TimerEvent_t *obj, void (*callback) (void))
```

2. Set the timer a timeout value on milliseconds

```
void TimerSetValue (TimerEvent_t *obj, uint32_t value)
```

3. Start the timer

```
void TimerStart (TimerEvent_t *obj)
```

4. Stop the timer

```
void TimerStop (TimerEvent_t *obj)
```

Hints:

User defined software timer can be used i.e. to control Tx and/or Rx led

I-CUBE-LRWAN basics: mlm32l0xx hw functions

There are functions related to hardware implemented in mlm32l0xx_hw.c / hw_msp.h

1. Get value of internal temperature sensor of STM32L072 in [°C]
`uint16_t HW_GetTemperatureLevel( void );`

2. Get battery level, 1 (very low) to 254 (fully charged)
`uint8_t HW_GetBatteryLevel( void );`

3. Low power modes management

4. Get board 64 bits unique ID

5. Peripherals management

I-CUBE-LRWAN basics: PRINTF redirection

PRINTF stream redirection to ST-Link Virtual Com Port was implemented in files vcom.c / vcom.h

1. Initialization of UART, default parameters: 115200,8,N,1
`void vcom_Init(void)`

```
UartHandle.Init.BaudRate      = 115200;  
UartHandle.Init.WordLength    = UART_WORDLENGTH_8B;  
UartHandle.Init.StopBits      = UART_STOPBITS_1;  
UartHandle.Init.Parity        = UART_PARITY_NONE;  
UartHandle.Init.HwFlowCtl     = UART_HWCONTROL_NONE;  
UartHandle.Init.Mode          = UART_MODE_TX_RX;
```

2. Redefined PRINTF() function can be used instead of printf() to output debug stream to Com Port terminal i.e.
`PRINTF("Temperature = %d\n", temperature);`

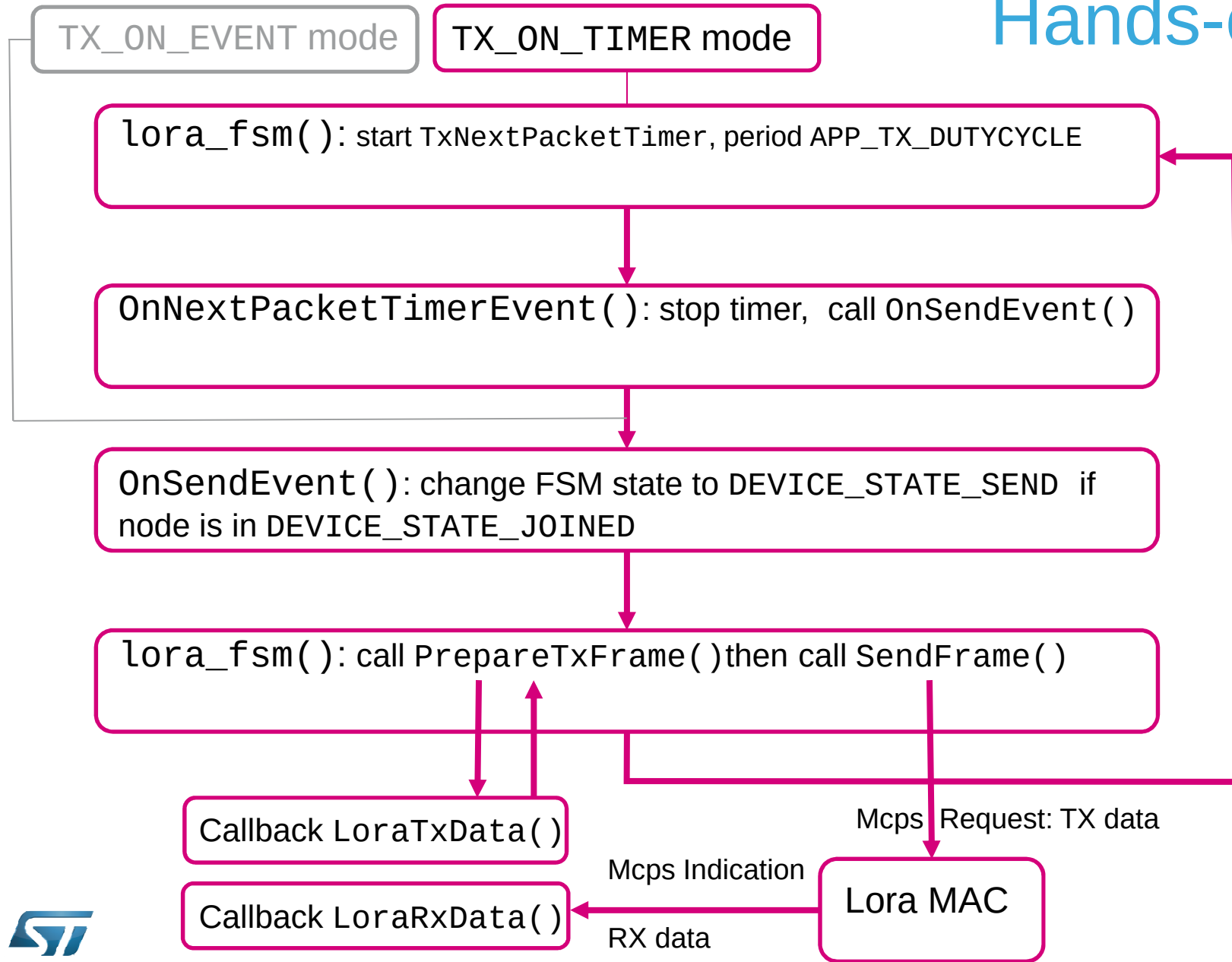
Hands-on

Implementation
How does it works ?

Hands-on

1. Lora mote sends data periodically to gateway according to TX_ON_TIMER initialization parameter, period is equal to APP_TX_DUTYCYCLE; main.c
2. All the process is handled by software timer TxNextPacketTimer and OnTxNextPacketTimerEvent() callback; lora.c
3. Uplink data send procedure is triggered by function OnSendEvent() called in timer callback; lora.c
4. FSM handles data send process by call PrepareTxFrame() and SendFrame() function, user can prepare own payload in LoraTxData() callback which is called by PrepareTxFrame() function; lora.c
5. Lora MAC takes control and call LoraRxData() callback, timing is handled by software timers as well; LoRaMac.c
6. It is possible to use TX_ON_EVENT mode and send uplink data asynchronously but OnSendEvent() function should be called in user defined callback function i.e. external interrupt

Hands-on



Hands-on

Lets start coding...

Hints:

Open related project

...\LORA_WS\Hands_on_v02\Projects\Multi\Applications\LoRa\End_Node_
temp_sensor\MDK-ARM\B-L072Z-LRWAN1

Hands-on

Modification of **main.c**

`/* Private define --- */ section`

1. Add definition of application data buffer size

```
#define APP_DATA_BUF_SIZE 4
```

2. Set definition of application data transmission duty cycle to 60s

```
#define APP_TX_DUTYCYCLE 45000
```

Hands-on

Modification of **main.c**

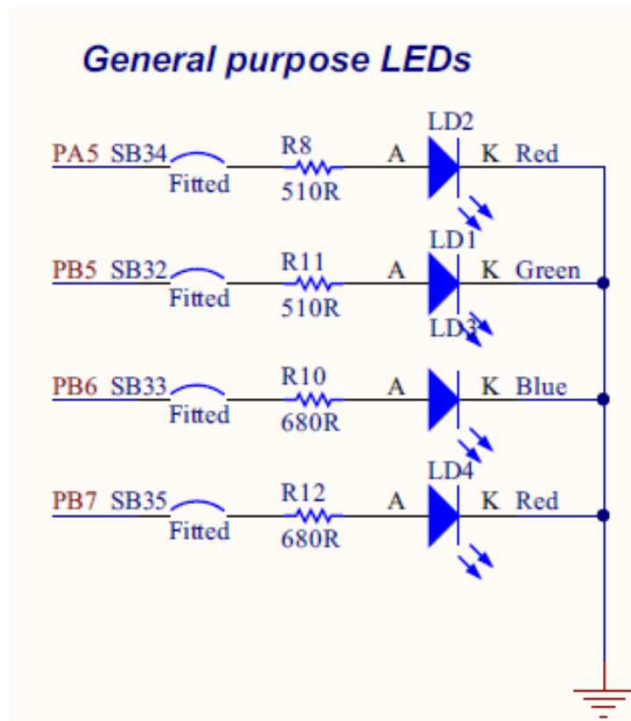
```
/* Private variables --- */ section  
/* Private functions --- */ section
```

1. Add software timer to control Tx LED
`static TimerEvent_t TxLedTimer;`
2. Add TxLedTimer callback prototype to control Tx LED
`static void OnTxLedTimerEvent( void );`

Hands-on

Modification of **main.c**

Pin assignment of general purpose LEDs of B-L072Z-LRWAN1 Discovery board



Tx LED definition

LED_RED1 - LD2 - PA5

Application LEDs definition

LED_GREEN - LD1 - PB5

LED_BLUE - LD3 - PB6

LED_RED2 - LD4 - PB7

Modification of **main.c**

Hands-on

USER CODE BEGIN / END section

During workshop we can face quite high RF traffic and due to collisions avoidance, random RF transmit period should be introduced

```
srand1(HW_GetRandomSeed());  
uint32_t rndTxPeriod = randr(APP_TX_DUTYCYCLE, APP_TX_DUTYCYCLE * 2);  
LoRaParamInit.TxDutyCycleTime = rndTxPeriod;  
PRINTF("\nTX period: %dms\n\n", rndTxPeriod);
```

Hands-on

Modification of **main.c** LoraTxData() callback

```
/* USER CODE BEGIN 3 */

int16_t temperature = 0;

TimerInit(&TxLedTimer, OnTxLedTimerEvent); //Tx led on
TimerSetValue(&TxLedTimer, 200); //Tx led on for 200ms
LED_On(LED_RED1);
TimerStart(&TxLedTimer);
temperature = HW_GetTemperatureLevel();
temperature = temperature >> 8; //Cut fractional part
PRINTF("Temperature = %d\n", temperature);

AppData->Port = LORAWAN_APP_PORT;
*IsTxConfirmed = LORAWAN_CONFIRMED_MSG;
AppData->BuffSize = snprintf((char *)AppData->Buff \
, APP_DATA_BUF_SIZE, "%d", temperature);

/* USER CODE END 3 */
```

Modification of **main.c** LoraRxData() callback

```
/* USER CODE BEGIN 4 */
if (AppData->Port==LORAWAN_APP_PORT)
{
    switch (AppData->Buff[0])
    {
        case 'E':
            LED_On(LED_GREEN); LED_Off(LED_BLUE); LED_Off(LED_RED2);
            PRINTF("Temperature OK.\n\n");
            break;
        case 'I':
            LED_Off(LED_GREEN); LED_On(LED_BLUE); LED_Off(LED_RED2);
            PRINTF("Temperature too LOW.\n\n");
            break;
        case 'D':
            LED_Off(LED_GREEN); LED_Off(LED_BLUE); LED_On(LED_RED2);
            PRINTF("Temperature too HIGH.\n\n");
            break;
        default:
            LED_Off(LED_GREEN); LED_Off(LED_BLUE); LED_Off(LED_RED2);
            PRINTF("Received data unknown. All LEDs Off.\n\n");
            break;
    }
    PRINTF("\n");
}
/* USER CODE END 4 */
```

Modification of **main.c** `OnTimerLedEvent()` callback

1. Add `OnTimerLedEvent()` callback to switch off Tx led when related software timer has expired

```
static void OnTxLedTimerEvent( void )  
{  
    LED_Off(LED_RED1);  
}
```

Hands-on

Modification of **lora.c** Duty cycle control feature

1. Switch off duty cycle control

```
#define LORAWAN_DUTYCYCLE_ON          false
```

Hints:

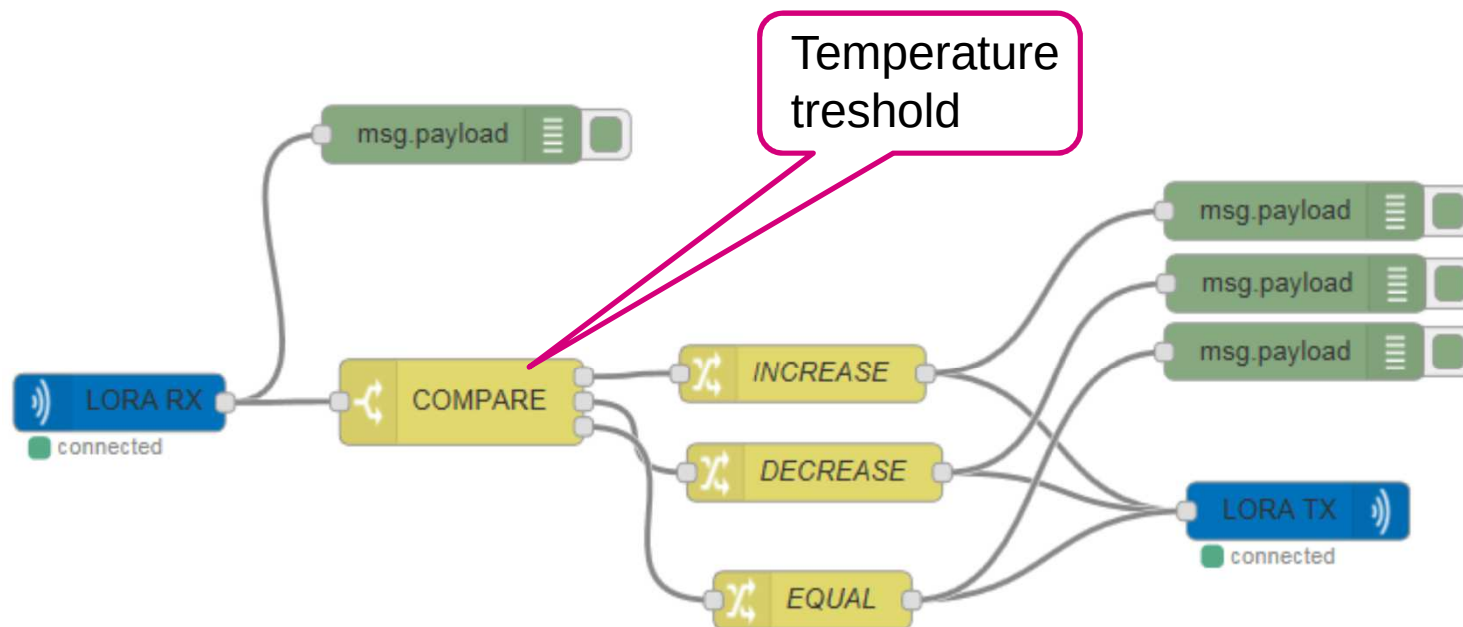
According to EN300220-1 there is a duty cycle limitation in ISM band: *“In a period of 1 hour the duty cycle shall not exceed the spectrum access and mitigation requirement values...”*. For 868MHz sub-band used by Lora it is 1%. LoraWAN protocol stack controls duty cycle but for our hands-on we can switch it off due to practical reasons (quite high on-air traffic).

Hands-on

Lets compile and download binary...

Hands-on

Application server functionality



Hands-on

Terminal output (115200,8,N,1) : joining

```
OTAA
DevEui= 34-31-37-32-7A-36-79-0D
AppEui= 01-01-01-01-01-01-01-01
AppKey= 2B 7E 15 16 28 AE D2 A6 AB F7 15 88 09 CF 4F 3C

VERSION: 44011000

INIT
JOINING...
txDone
rxDone
JOINED
Temperature = 27
txDone
rxDone
Temperature OK.
```

Lora mote already joined

Green Tx led blinks

Hands-on

Terminal output (115200,8,N,1) : sending temperature

```
Temperature = 28  
txDone  
  
rxDone  
Temperature too HIGH. ← Red led on  
  
[00]Temperature = 27  
txDone  
  
rxDone  
Temperature OK. ← Green led on  
  
[00]Temperature = 27  
txDone  
  
rxDone  
Temperature OK. ← Green led on  
  
[00]Temperature = 28  
txDone  
  
rxDone  
Temperature too HIGH. ← Red led on
```

Hands-on

Thank you